



Escuela de Ingeniería en Computadores

CE1106 — Paradigmas de programación

Profesor: Marco Rivera Meneses

Documentación Técnica de Tarea 4

Estudiantes:

Jian Yong Zheng Wu

jiazheng@estudiantec.cr

2023058713

Yitzy Jiménez Jiménez

Yitzy.jimenez1726@estudiantec.cr

2021018775

Jose Fernández Rojas

jo.fernandez@estudiantec.cr

2022180283

Fecha: November 28, 2025

1 Descripción de las estructuras de datos

1.1 Servidor Java

El diseño en Java sigue un enfoque orientado a objetos y se organiza por paquetes:

- **Paquete `com.dkj.model`:** contiene tipos de apoyo como `Position`, `Height`, `LianaId`, `Platform`, `PlayerId`, `Points`, `Speed`, `Direction`, entre otros. Estos tipos encapsulan datos y evitan el uso excesivo de tipos primitivos.
- **Paquete `com.dkj.entities`:**
 - `Entity`: interfaz base para entidades del mundo.
 - `Player`: representa a Donkey Kong Jr.
 - `CrocodileRed`: cocodrilo que sube y baja en una misma liana.
 - `CrocodileBlue`: cocodrilo que desciende y termina cayendo.
 - `Fruit`: fruta que otorga puntos.
 - `DefaultEntityFactory`: fábrica concreta para crear jugadores, cocodrilos y frutas.
- **Paquete `com.dkj.game`:**
 - `Game`: mantiene las colecciones de jugadores, enemigos y frutas y coordina la lógica principal del juego.
 - `PhysicsEngine`: se encarga de la física y del manejo de los *inputs* de movimiento recibidos desde la red.
 - `GameLoop`: ejecuta actualizaciones periódicas del juego.
 - `Level`: describe la geometría del nivel y posiciones iniciales de los elementos importantes.
- **Paquete `com.dkj.events`:**
 - Interfaces `GameEvent` y `GameEventListener`.
 - Eventos concretos como `StateEvent`, `ScoreEvent`, `DeathEvent`, `LevelEvent`.
 - `GameEventBus`: implementa un bus de eventos que notifica a todos los oyentes registrados.
- **Paquete `com.dkj.server`:**
 - `LineServer`: servidor TCP de líneas de texto.
 - `ClientContext`: contiene información de cada cliente conectado (identificador, rol, canal de escritura).
 - `SessionRegistry`: registro de sesiones activas.
 - `MatchRegistry`: administra las salas de juego y el límite de jugadores.
 - `GameRoom`: instancia un `Game` y se encarga de reenviar eventos a jugadores y espectadores.

- **Paquete `com.dkj.admin`:**
 - **AdminConsole**: consola de texto para enviar comandos de administración.
 - **AdminWindow**: ventana gráfica para administración (opcional).

1.2 Cliente en C

En C se usan *structs* para representar el estado del juego y del nivel:

- **PlayerState**: contiene el id del jugador, posición, velocidad, vidas, puntaje y banderas como `onGround` y `onLiana`.
- **RedCroc** y **BlueCroc**: guardan la liana asociada, la posición y la velocidad vertical de los cocodrilos rojos y azules respectivamente.
- **FruitState**: liana, altura y puntos otorgados de cada fruta activa.
- **Platform** y **Liana**: información geométrica usada para colisiones y detección de agarre.
- **Level**: arreglos de plataformas, lianas y posiciones relevantes del mapa (posición de Donkey Kong, Mario, llave, etc.).
- **World**: *snapshot* del estado completo del mundo recibido desde el servidor; contiene arreglos de jugadores, cocodrilos y frutas, junto con sus contadores.

Todas las constantes globales se definen en `constants.h` (host, puerto, límites lógicos del nivel, tamaños de sprites y otros valores reutilizados), mientras que la lógica geométrica se implementa en `level.c` y la comunicación de red en `network.c`.

2 Descripción detallada de los algoritmos

2.1 Loop de juego en el servidor

El `GameLoop` utiliza un `ScheduledExecutorService` para ejecutar actualizaciones periódicas. En cada *tick* se llama a `Game.step()`, que realiza, de forma resumida, lo siguiente:

1. Leer los comandos de movimiento pendientes gestionados por **PhysicsEngine** (cola de `MoveCommand`).
2. Actualizar posición y velocidad de los jugadores, verificando colisiones con plataformas y lianas.
3. Actualizar posición de cocodrilos rojos y azules según su velocidad y dirección.
4. Detectar colisiones con frutas y enemigos, ajustando el puntaje o las vidas según corresponda.
5. Generar eventos de estado, puntos, muerte o cambio de nivel y publicarlos en el `GameEventBus`.

La `GameRoom` está suscrita al bus de eventos y traduce estos eventos en mensajes de texto que se envían a jugadores y espectadores.

2.2 Movimiento de cocodrilos y frutas

Cocodrilo rojo. Se mueve de forma vertical en una sola liana. Mantiene una altura continua y una dirección (subiendo o bajando). Cuando llega a los límites definidos rebota y cambia la dirección de movimiento, de forma que patrulla un rango fijo de la liana.

Cocodrilo azul. Se crea en una liana y altura inicial y se desplaza hacia abajo con velocidad constante. Al salir del rango válido del nivel se considera que ha caído y se desactiva del juego.

Frutas. Cada fruta tiene liana, altura y puntos. Cuando la posición del jugador coincide con la liana y altura de una fruta dentro de un margen tolerado, se suman los puntos al jugador y la fruta se elimina del estado del juego.

2.3 Colisiones y nivel en el cliente C

En `level.c` se implementan funciones para la geometría y colisiones:

- `level_check_ground`: revisa si el jugador está sobre alguna plataforma y actualiza el estado de si está en el suelo o en caída.
- `level_find_nearest_liana`: busca la liana más cercana a la coordenada horizontal del jugador.
- `level_can_grab_liana`: determina si el jugador puede agarrarse de una liana dada su posición en el mapa.
- Funciones auxiliares para detectar la recogida de la llave y la llegada a la plataforma de Donkey Kong (condiciones de victoria).

Estas funciones trabajan en conjunto con el estado del `World` para decidir cómo dibujar al jugador y cuándo cambiar entre correr, saltar o colgarse de una liana.

2.4 Protocolo de red y sincronización

El protocolo de red utiliza líneas de texto sobre TCP. Algunos mensajes desde el cliente son:

- `HELLO PLAYER`
- `HELLO SPECTATOR <id>`
- `MOVE <UP|DOWN|LEFT|RIGHT|NONE>`
- `BYE`

El servidor responde con confirmaciones (`ACK`) o errores (`ERR`) y envía mensajes de estado que permiten reconstruir el `World` en el cliente.

En C, `network.c` encapsula el uso de WinSock2 y emplea un hilo separado para leer mensajes del servidor. El bucle de Raylib se limita a dibujar y procesar entrada de usuario, consultando el estado actualizado por el hilo de red.

2.5 Aumento de dificultad al ganar

Cuando el jugador llega hasta Donkey Kong:

1. El servidor detecta la victoria a partir de la posición del jugador en el nivel.
2. Se emite un evento de cambio de nivel y se reinicia el estado del juego.
3. Se incrementa la velocidad de los cocodrilos rojos y azules para aumentar la dificultad.
4. Se otorga una vida adicional al jugador antes de reiniciar la partida.

3 Patrones de diseño aplicados

En el desarrollo del servidor se aplicaron varios patrones de diseño, en este caso implementamos el Abstract Factory, el Observer y el Command.

Abstract Factory: La clase `DefaultEntityFactory` centraliza la creación de objetos del mundo del juego (jugadores, cocodrilos rojos, cocodrilos azules y frutas). A partir de parámetros lógicos como tipo de entidad, liana, altura o puntos, la fábrica construye las instancias correspondientes. De esta forma, el código de alto nivel que administra las partidas solo solicita entidades a la fábrica y no depende de los constructores concretos ni de detalles internos de cada clase.

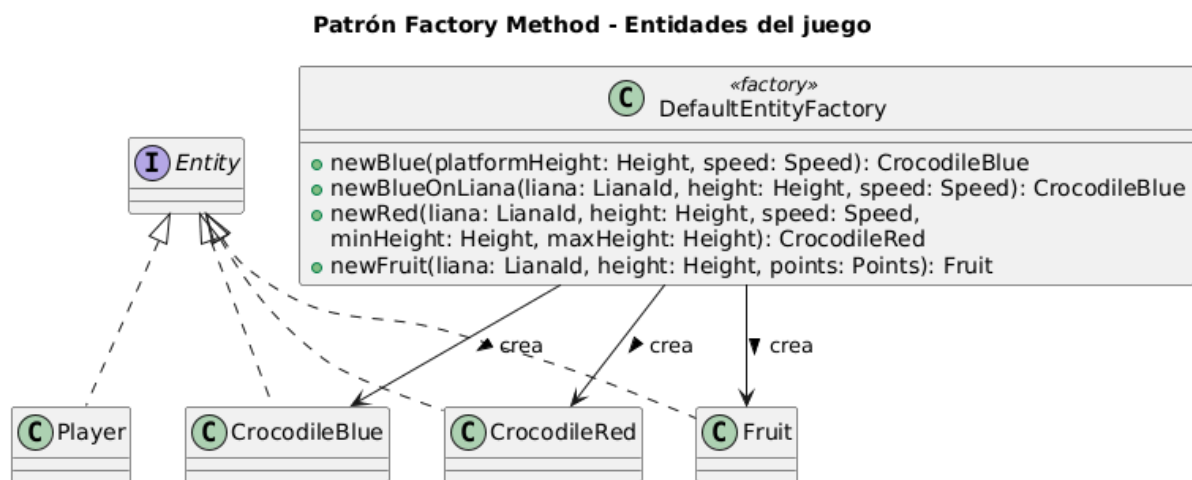


Figure 1: Diagrama de Clases del Patrón Abstract Factory.

Observer: El módulo de eventos (`GameEventBus`, `GameEvent` y `GameEventListener`) implementa el patrón *Observer*. El juego genera eventos cuando cambian el estado, el puntaje o la vida del jugador, y dichos eventos se notifican a todos los observadores registrados. Entre los observadores se encuentran, por ejemplo, las `GameRoom` que reenvían la información a los clientes jugadores y espectadores.

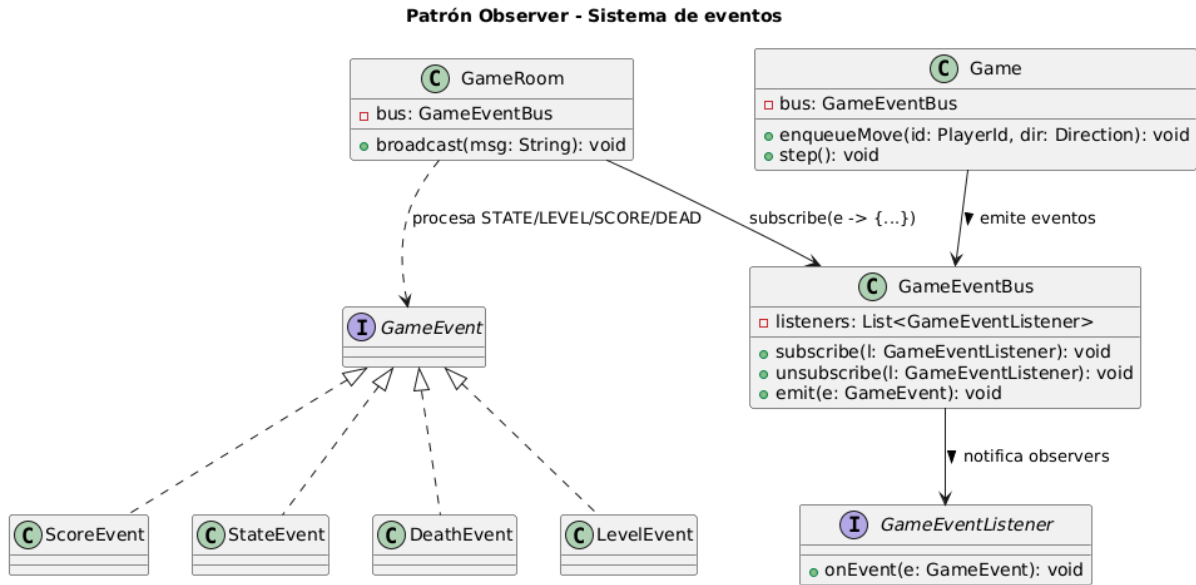


Figure 2: Diagrama de Clases del Patrón Observer.

Command: El manejo de comandos de texto que llegan por la red (MOVE, HELLO, ADMIN, etc.) se organiza con un despachador de comandos (`CommandDispatcherWithGame`) que encapsula cada acción en un manejador independiente. Esto simplifica el código del servidor y facilita agregar o modificar comandos sin afectar el resto de la lógica del juego.

Patrón Command - Comandos de red y movimientos

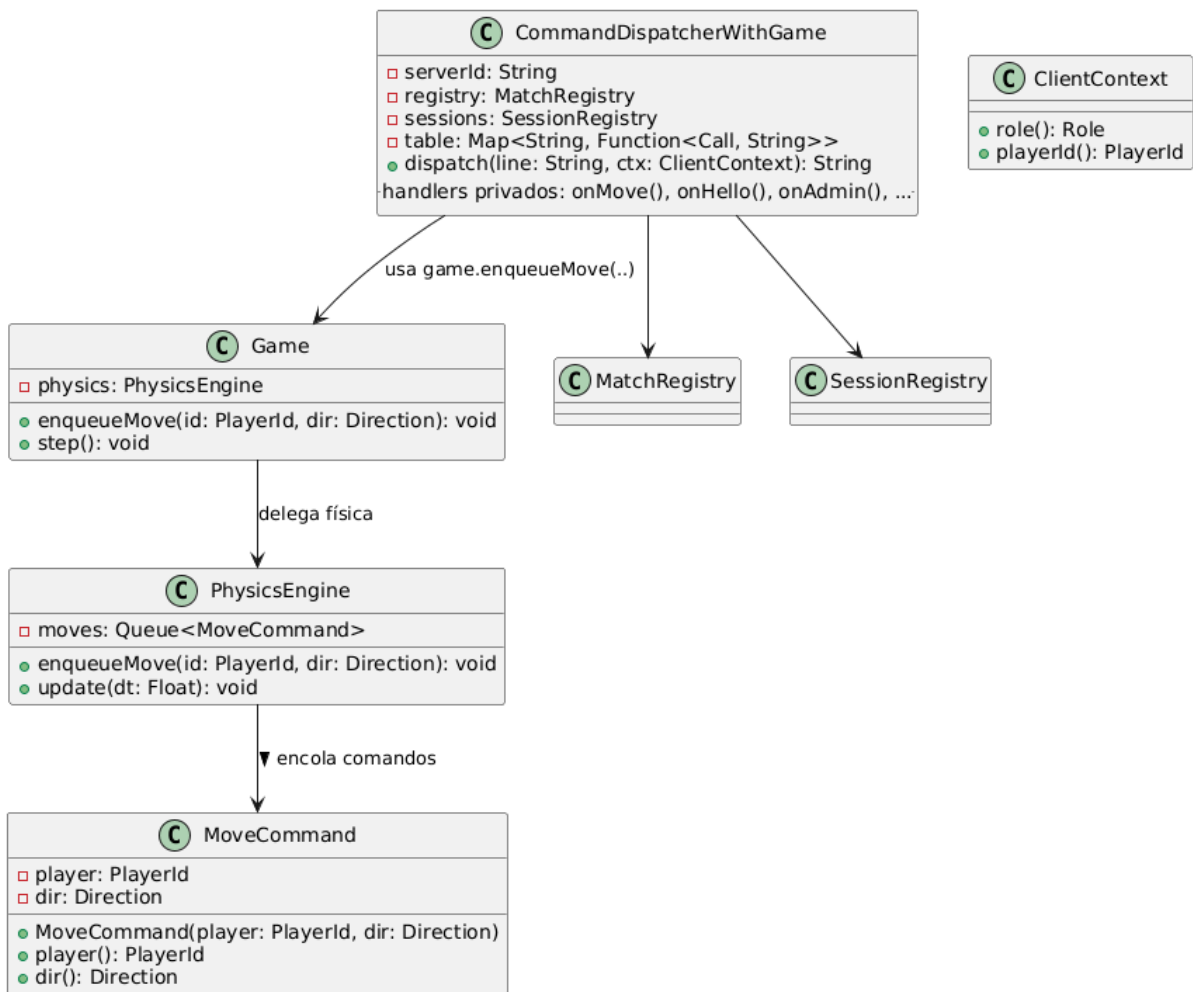


Figure 3: Diagrama de Clases del Patrón Command.

4 Problemas sin solución

En la versión actual del proyecto quedaron algunos puntos pendientes:

- No se implementó persistencia de puntajes ni guardado de partidas; cada vez que se reinicia el servidor se pierde el historial.
- No existe un mecanismo de reconexión automática si un cliente pierde la conexión; es necesario volver a ejecutar el cliente manualmente.
- Los espectadores solo pueden observar el estado actual de la partida; no hay forma de ver el inicio de una partida que ya avanzó.

5 Problemas encontrados

5.1 Bloqueo de la ventana por operaciones de red

Al inicio, la recepción de datos de red se hacía en el mismo hilo que el render de Raylib, lo que generaba congelamientos cuando el servidor tardaba en responder o cuando se

producían ciertos errores de conexión.

La solución fue crear un hilo dedicado a la recepción de mensajes y definir *callbacks* para procesarlos. El bucle de Raylib consulta el estado ya actualizado por ese hilo, evitando llamadas bloqueantes dentro del render y haciendo que la interfaz gráfica se mantenga fluida.

5.2 Manejo de comandos de administración

La sintaxis de los comandos ADMIN para creación y eliminación de cocodrilos y frutas es relativamente compleja y propensa a errores si se parsea a mano con estructuras *if/else* muy largas.

Se implementó un despachador de comandos que centraliza el parseo y la validación de parámetros. Cada comando se mapea a un manejador específico, lo cual simplificó el código, facilitó el manejo de errores y deja el sistema listo para agregar nuevos comandos en el futuro.

5.3 Límite de jugadores y espectadores

El enunciado exige un máximo de dos jugadores activos y dos espectadores por jugador. Para cumplirlo sin mezclar responsabilidades, se definieron constantes y métodos específicos en *MatchRegistry* y *GameRoom* que verifican la capacidad antes de aceptar una nueva conexión.

Esto ayudó a mantener claro qué parte del código se encarga de las sesiones y cuál administra las partidas, evitando inconsistencias entre ambos módulos.

6 Conclusiones

- Se logró separar de forma clara el uso del paradigma imperativo en C (cliente gráfico y gestión de memoria manual) y del paradigma orientado a objetos en Java (servidor, entidades y patrones).
- El uso de *raylib* permitió crear una interfaz cercana al estilo del juego original, con manejo de sprites, HUD y animaciones simples.
- La combinación de eventos (Observer), fábricas de entidades (Abstract Factory) y un despachador de comandos (Command) simplificó la comunicación entre la lógica del juego y los clientes.
- Trabajar con sockets y concurrencia en ambos lenguajes reforzó conceptos de programación concurrente y de red, así como la importancia de separar la lógica de juego de la capa de interfaz.

7 Recomendaciones

- Agregar persistencia de puntajes y estadísticas en archivos o en una base de datos, para conservar el progreso de las partidas.
- Mejorar el manejo de errores de red, incluyendo intentos de reconexión automática y mensajes más detallados para el usuario final.

- Permitir configurar parámetros del nivel (número de lianas, velocidad base, gravedad, etc.) desde la interfaz de administración.
- Realizar refactorizaciones si se agregan nuevos tipos de enemigos o *power-ups*, aprovechando las fábricas y el sistema de eventos ya implementados.

Bibliografía

- [1] OpenAI. (2025) Chatgpt. [Online]. Available: <https://chat.openai.com/>.
- [2] Stack Overflow. (2025) Stack overflow. [Online]. Available: <https://stackoverflow.com/>.
- [3] DeepSeek. (2025) Deepseek. [Online]. Available: <https://www.deepseek.com/>.